

A Hybrid Two-Stage Framework for Twitter Bot Detection: Decoupling Graph Representation Learning and Classification

Khoi Nguyen Pham^{1,2}, Ngoc Thao Vy Tran^{1,2}, Thanh Quan Nguyen^{1,2},
Kim Ngan Tran^{1,2}, Minh Phuong Ha^{1,2}, and Hung-Nghiep Tran^{1,2,*}

¹University of Information Technology, Ho Chi Minh City, Vietnam

²Vietnam National University, Ho Chi Minh City, Vietnam

Email: {23521055, 23521836, 23521266, 23520999, 23521241}@gm.uit.edu.vn, nghiepth@uit.edu.vn

Abstract—Graph-based Twitter bot detection models such as BotRGCN rely on end-to-end training that jointly optimizes graph representation learning and linear classification. This coupling creates a bottleneck: the GNN is constrained to produce linearly separable representations, and joint optimization restricts the classifier to differentiable models. This study proposes a hybrid two-stage framework that decouples these two components. In Stage 1, BotRGCN serves as a graph-aware feature extractor. In Stage 2, XGBoost is trained independently on the frozen embeddings, a non-differentiable classifier only accessible through decoupling. Across 5 random seeds on TwiBot-22, where severe class imbalance and million-scale heterogeneous structure make each F1 score reflect substantial discriminative gain, the proposed method achieves F1 of 60.41 (+2.81 over the BotRGCN baseline) while reducing cross-seed variance from 0.78 to 0.26. The improvement is consistent across GCN, GAT, and BotRGCN architectures, suggesting that coupling, rather than architecture choice, may be a significant bottleneck. Classifier ablation further shows that even Logistic Regression on frozen embeddings outperforms end-to-end training, indicating that decoupling serves as a key mechanism.

Index Terms—Twitter bot detection, graph neural networks, decoupled learning, XGBoost, TwiBot-22

I. INTRODUCTION

Social media platforms have become prime targets for automated malicious accounts, or social bots, capable of spreading misinformation and manipulating online discourse at scale [1], [2], affecting the results of political elections [3], and undermining the integrity of online social networks [4]. Graph-based approaches represent online social networks as heterogeneous graphs, where users are nodes and interactions such as following, retweeting, and mentioning form edges. Graph Neural Networks (GNNs) exploit this structure by aggregating neighborhood information, allowing the model to capture collective bot behaviors [5]. For instance, BotRGCN [6] encodes multi-modal user features, including profile metadata, tweet content, and relational graph structure through relational graph convolutional layers, achieving strong performance on large-scale benchmarks such as TwiBot-22 [7].

Despite their effectiveness, existing GNN-based methods share a common architectural pattern: representation learning and classification are jointly optimized in an end-to-end pipeline. Specifically, the GNN encoder and the classification

head are trained simultaneously using a single loss function, where gradients from the classification objective flow back through the entire network. This creates competing objectives: the GNN must simultaneously learn representations that are structurally informative for graph propagation and linearly separable for the classification head. These two objectives are not always aligned, and optimizing for one may come at the cost of the other. Furthermore, joint optimization restricts the classifier to differentiable models, excluding powerful non-differentiable alternatives such as gradient boosted trees. To address this bottleneck, this study proposes decoupling the classification stage from GNN training, enabling the use of non-differentiable classifiers such as XGBoost [8]. As a gradient boosted tree ensemble, XGBoost cannot be integrated into an end-to-end pipeline. Thus, decoupling is not a design choice but a technical necessity. Moreover, gradient boosted trees are known to perform well on structured, moderate-dimensional feature spaces [9], which match the characteristics of the 128-dimensional GNN embeddings extracted by BotRGCN.

This study proposes a hybrid two-stage framework that explicitly decouples these two concerns. In Stage 1, BotRGCN is trained to convergence as a pure feature extractor. In Stage 2, XGBoost is trained independently on the frozen embeddings. Across 5 random seeds on TwiBot-22 [7], which contains 1,000,000 accounts with severe class imbalance (14% positive class), the proposed method achieves F1 of 60.41, a +2.81 improvement over the BotRGCN baseline. This margin represents a meaningful discriminative gain, while cross-seed variance is simultaneously reduced from 0.78 to 0.26. The consistent improvement across GCN [10], GAT [11], and BotRGCN architectures indicates that the bottleneck likely stems from the coupling rather than the architecture. The main contributions of this paper are:

- A two-stage framework that decouples GNN representation learning from classification, enabling the use of non-differentiable classifiers such as XGBoost.
- Consistent improvement across three GNN architectures (+2.11 to +4.74 F1) with reduced variance, supporting decoupling as a contributing factor in the observed gains.

- A simple, effective, and stable two-stage framework validated through comprehensive ablation studies across classifiers, embedding strategies, and XGBoost hyperparameters.

II. RELATED WORK

Early bot detection methods rely on feature engineering, extracting handcrafted features from user metadata and tweet content [4], [12]. With the rise of deep learning, text-based approaches leverage language models to analyze user-generated content. More recently, graph-based methods have emerged as the dominant paradigm, modeling online social networks as heterogeneous graphs and applying GNNs to aggregate relational information [12]. Representative works include BotRGCN [6], which applies relational graph convolution on follower/following relationships, and HCGBot [13], which learns homophilous context graphs for improved detection, reporting F1 of 62.25 on TwiBot-22. While decoupling strategies have been explored in GNN literature to separate feature propagation from transformation [14], their application to bot detection remains unexplored. In GNN-based anomaly detection, Wang et al. [15] show that decoupling the encoder from the classifier reduces bias from hard instances and outperforms joint training. Similarly, Kang et al. [16] demonstrate in long-tailed recognition that retraining only the classifier on frozen representations outperforms end-to-end optimization under severe class imbalance, a setting analogous to TwiBot-22’s 14% positive class distribution. Beyond GNN literature, using pretrained deep networks as fixed feature extractors for classical classifiers has been explored in vision tasks [17]. To the best of our knowledge, no prior work has applied decoupled learning to bot detection with non-differentiable classifiers. This work addresses that gap by operating on graph embeddings produced by relational message passing.

III. PROBLEM FORMULATION

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{R})$ be a heterogeneous social graph where $\mathcal{V}$ is the set of users, $\mathcal{E}$ the set of edges, and $\mathcal{R} = \{\text{following}, \text{follower}\}$ the relation types. Each user $v \in \mathcal{V}$ is associated with description B , tweets T , and user properties $P = \{P^{num}, P^{cat}\}$. The task is to learn $f : \mathcal{V} \rightarrow \{0, 1\}$ where 1 denotes a bot. In the standard end-to-end paradigm, a GNN encoder h_θ and linear classifier g_ϕ are jointly optimized via a single objective $\mathcal{L}(g_\phi(h_\theta(\mathcal{G})), \mathbf{y})$. This coupling constrains h_θ to produce linearly separable representations. The two-stage framework instead trains h_θ first, extracts frozen embeddings $\mathbf{E} = h_{\theta^*}(\mathcal{V}) \in \mathbb{R}^{|\mathcal{V}| \times d}$, then trains XGBoost independently on $\mathbf{E}$ - a non-differentiable classifier only accessible through decoupling.

IV. MATERIALS AND METHODS

A. Dataset

This study uses TwiBot-22 dataset [7], the largest graph-based Twitter bot detection benchmark, containing approximately 1 million labeled accounts (860,057 human, 139,943 bot). The dataset is structured as a heterogeneous social graph

with 4 entity types and 14 relation types, enabling multi-relational graph learning. The official split is followed throughout all experiments.

B. Experiment Settings

BotRGCN is trained using the AdamW optimizer with learning rate 1×10^{-2} and weight decay 5×10^{-2} . Dropout is set to 0.3 and embedding dimension to 64. XGBoost is configured with 300 estimators, a maximum depth of 6, a learning rate of 0.1, row subsampling of 0.8, column subsampling of 0.8, and the evaluation metric log loss; all other parameters are left at default. These hyperparameters were fixed prior to all experiments and were not tuned on the test set; the hyperparameter sweep in Section V-D serves solely as a post-hoc sensitivity analysis to demonstrate robustness. All results are reported as mean $\pm$ std over 5 random seeds from 42 to 46 on the official TwiBot-22 test split, and binary F1-score is used as the primary evaluation metric.

C. Proposed Framework

Following [6], BotRGCN encodes four user modalities: description and tweet content via RoBERTa [18] (768-dim each), numerical properties $P^{num} \in \mathbb{R}^5$, and categorical properties $P^{cat} \in \mathbb{R}^3$. Each modality is projected to $d/4$ dimensions and concatenated to form the initial node representation $\mathbf{x}_v \in \mathbb{R}^d$:

$$\mathbf{x}_v = [\mathbf{r}_B \| \mathbf{r}_T \| \mathbf{r}_{num} \| \mathbf{r}_{cat}] \quad (1)$$

Node representations are then refined through two R-GCN layers [19]:

$$\mathbf{x}_v^{(l+1)} = \mathbf{W}_0 \mathbf{x}_v^{(l)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{|\mathcal{N}_r(v)|} \mathbf{W}_r \mathbf{x}_u^{(l)} \quad (2)$$

After convergence, h_{θ^*} is frozen and node embeddings are extracted for all users. Each R-GCN layer produces a 64-dimensional embedding; concatenating both layers yields a 128-dimensional representation, forming $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times 128}$, on which XGBoost is trained independently using only the training split:

$$\hat{y}_v = \text{XGBoost}(\mathbf{e}_v), \quad \mathbf{e}_v \in \mathbf{E} \quad (3)$$

Figure 1 illustrates the complete pipeline.

D. Ablation Design

To systematically evaluate the two-stage framework, three ablation studies are conducted. First, four classifiers are compared on frozen BotRGCN embeddings $\mathbf{E}$: XGBoost, Logistic Regression (LR), Random Forest (RF), and Multilayer Perceptron (MLP).

$\hat{y}_v = f(\mathbf{e}_v)$, $f \in \{\text{XGBoost}, \text{LR}, \text{RF}, \text{MLP}\}$, $\mathbf{e}_v \in \mathbf{E}$ (4) where $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times 128}$ are the frozen concatenated embeddings and f is varied while $\mathbf{E}$ is held fixed. Logistic Regression serves as a near-equivalent to BotRGCN’s own linear head, allowing performance gains to be attributed to decoupling rather than classifier complexity. Three embedding strategies s are evaluated:

$$\mathbf{e}_v^{(s)} = \begin{cases} \mathbf{x}_v^{(1)} & s = \text{first}, & \mathbf{e}_v^{(s)} \in \mathbb{R}^{64} \\ \mathbf{x}_v^{(2)} & s = \text{last}, & \mathbf{e}_v^{(s)} \in \mathbb{R}^{64} \\ [\mathbf{x}_v^{(1)} \| \mathbf{x}_v^{(2)}] & s = \text{concat}, & \mathbf{e}_v^{(s)} \in \mathbb{R}^{128} \end{cases}$$

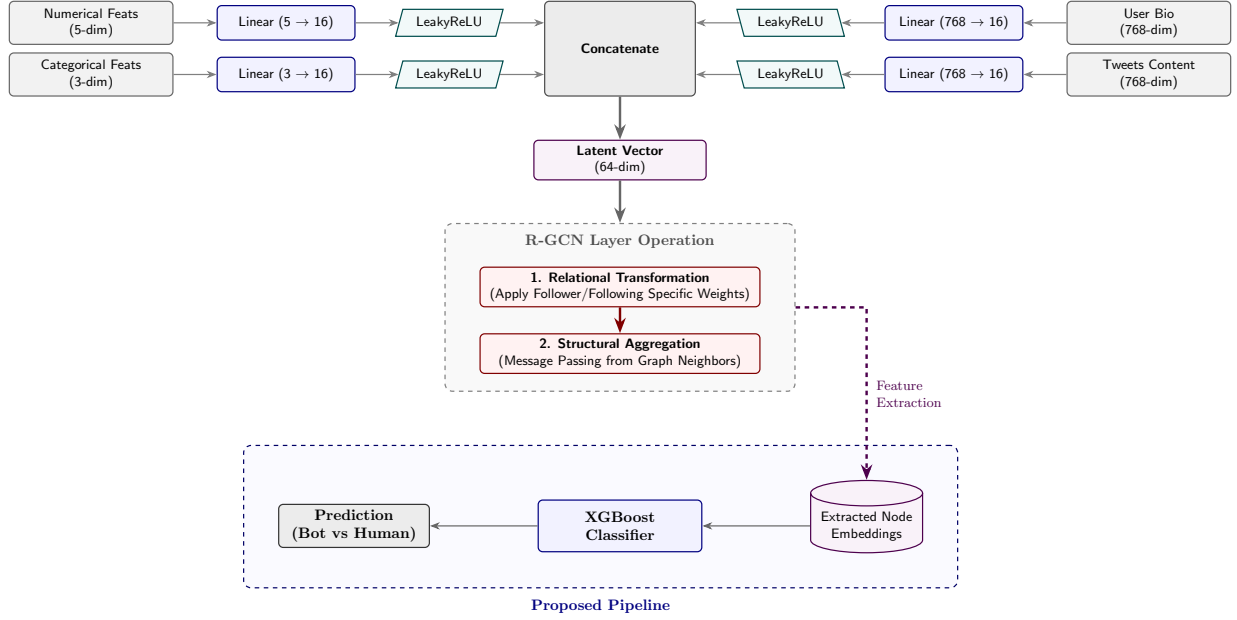


Fig. 1: Overview of the proposed two-stage framework.

where $\mathbf{x}_v^{(1)}$ and $\mathbf{x}_v^{(2)}$ are the node representations from R-GCN layers 1 and 2 respectively, and the downstream classifier is fixed to XGBoost:

$$\hat{y}_v = \text{XGBoost}(\mathbf{e}_v^{(s)}) \quad (5)$$

A single-parameter sweep is conducted over six hyperparameters. At each evaluation, one parameter λ_k is varied over its candidate grid $\mathcal{G}_k$ while all others are held at default values:

$$\hat{y}_v = \text{XGBoost}(\mathbf{e}_v \mid \lambda_{\text{default}} \oplus \{\lambda_k = c\}), \quad c \in \mathcal{G}_k \quad (6)$$

where λ_{default} is the default configuration and $\oplus$ denotes substitution of a single parameter. The six parameter grids are:

$$\mathcal{G}_k \in \begin{cases} [100, 200, 300, 500, 800] & \text{n_estimators} \\ [3, 4, 6, 8, 10] & \text{max_depth} \\ [0.01, 0.05, 0.1, 0.2, 0.3] & \text{learning_rate} \\ [0.5, 0.6, 0.7, 0.8, 0.9, 1.0] & \text{subsample} \\ [0.5, 0.6, 0.7, 0.8, 0.9, 1.0] & \text{colsample_bytree} \\ [0.1, 0.5, 1.0, 2.0, 5.0] & \text{reg_lambda} \end{cases} \quad (7)$$

corresponding to number of estimators, maximum tree depth, learning rate, row subsampling ratio, column subsampling ratio, and L2 regularization strength, respectively.

V. RESULTS

A. Main Results: Decoupling

Table I compares the end-to-end GNN baseline (trained with the original softmax head) against the decoupled framework using XGBoost as the downstream classifier.

The decoupled framework improves F1 over the end-to-end baseline across all three architectures, with gains of +2.81 (BotRGCN), +4.74 (GAT [11]), and +2.11 (GCN [10]). Importantly, the variance reduction is as striking as the mean

TABLE I: End-to-end vs. decoupled XGBoost.

Model	End-to-End F1	Decoupled F1	ΔF1
GAT	54.58 ± 1.34	59.32 ± 0.26	+4.74
GCN	56.40 ± 0.91	58.51 ± 0.29	+2.11
BotRGCN	57.60 ± 0.78	60.41 ± 0.26	+2.81

improvement. The standard deviation decreases from 0.78 (end-to-end) to 0.26 (decoupled). GAT, whose end-to-end training exhibited the highest variance of ± 1.34 , achieves the tightest spread under decoupling of ± 0.26 . GCN similarly benefits, with standard deviation dropping from ± 0.91 to ± 0.29 under decoupling. These results suggest how separating the representation learning objective from the classification objective can stabilize the gradient signal during GNN training, regardless of the downstream classifier applied to the learned embeddings. The decoupled framework achieves a ROC-AUC of 0.7951 and PR-AUC of 0.6626, substantially above the random baseline (PR-AUC = 0.294, reflecting the 29% positive-class ratio in the test split), reflecting strong discriminative ability despite class imbalance in TwiBot-22 (Figure 3). The decoupled framework shifts the precision-recall trade-off toward higher recall compared to end-to-end baselines. While end-to-end BotRGCN achieves higher precision (66.46) at the cost of recall (50.89), the decoupled framework achieves 52.59 precision and 70.95 recall. For bot detection, where missing a malicious account carries higher risk than a false positive, this trade-off is generally favorable [20]. Furthermore, decoupling provides explicit control over this trade-off through the classifier's class weighting mechanism, which is not easily achievable in end-to-end training. A paired t -test across 5 seeds confirms the improvement is statistically significant ($t \approx 7.75$, $p \approx 0.0015$).

TABLE II: Classifier ablation on BotRGCN embeddings.

Classifier	Precision	Recall	F1
RF	71.68 $\pm$ 0.49	42.35 $\pm$ 1.18	53.23 $\pm$ 0.86
MLP	67.32 $\pm$ 0.54	50.37 $\pm$ 1.32	57.61 $\pm$ 0.75
LR	51.11 $\pm$ 0.55	72.72 $\pm$ 0.79	60.02 $\pm$ 0.40
XGBoost	52.59 $\pm$ 0.42	70.95 $\pm$ 0.51	60.41 $\pm$ 0.26

[†]XGBoost is adopted as the default classifier.

B. Classifier Ablation

Table II presents results across four classifiers applied to frozen BotRGCN embeddings. XGBoost achieves the best F1 of 60.41, followed by Logistic Regression with 60.02. While Random Forest falls below the end-to-end baseline with a mean F1 of 53.23, MLP achieves 57.61, roughly matching the end-to-end baseline but failing to benefit meaningfully from decoupling. The most notable finding is that Logistic Regression with F1 = 60.02 outperforms the end-to-end baseline despite being a linear classifier of the same functional family as BotRGCN’s own softmax head. This controls for the non-linearity hypothesis: if XGBoost’s gains were driven purely by its capacity to model non-linear decision boundaries, a linear classifier operating on the same frozen representations would be expected to match or underperform the end-to-end model. Random Forest’s high variance of ± 0.86 alongside its low mean reflects inconsistent generalization across seeds, indicative of overfitting to majority-class patterns in the embedding space.

C. Embedding Layer Ablation

TABLE III: Embedding layer ablation.

Embedding	Precision	Recall	F1
Last layer	52.19 $\pm$ 0.41	71.47 $\pm$ 0.63	60.33 $\pm$ 0.29
First layer	52.08 $\pm$ 0.44	71.79 $\pm$ 0.84	60.36 $\pm$ 0.21
All layers	52.59 $\pm$ 0.42	70.95 $\pm$ 0.51	60.41 $\pm$ 0.26

[†]All layers are adopted as the default configuration.

Table III demonstrates minimal performance variation across the three configurations. The mean F1 scores are nearly identical: 60.33 for the last layer, 60.36 for the first layer, and 60.41 for all layers concatenated. All differences fall within 0.1 F1, with standard deviations ranging from 0.21 to 0.29, indicating that these variations are within the normal range of experimental noise. This finding suggests that the decoupled classifier is robust to the choice of which layer representation is used as the embedding. Consequently, this study uses the all-layer concatenated embedding as the default configuration for all subsequent experiments.

D. XGBoost Hyperparameter Analysis

Figure 3 presents the ROC curve (AUC = 0.7951) and Precision-Recall curve (AUC = 0.6626) for the decoupled BotRGCN and XGBoost framework, confirming strong discriminative ability well above the random baseline despite class imbalance in TwiBot-22 dataset. Figure 2 shows the effect of six key hyperparameters on F1. The number of estimators shows a consistent upward trend, with F1 improving from 60.24 at 100 trees to 60.53 at 800 trees and variance decreasing steadily. Tree depth peaks at 6 before degrading at depth 10 with high

variance (± 0.48). Learning rate shows a clear optimum at 0.1, with performance dropping on both sides. Subsampling ratio, column subsampling, and L2 regularization all show minimal impact on F1, remaining stable across all tested values, suggesting that XGBoost is robust to these parameters on this task.

E. Class Weighting

TABLE IV: Class weighting analysis.

Setting	Precision	Recall	F1
<i>Class Weighting</i>			
Without	70.07 $\pm$ 0.53	46.61 $\pm$ 0.87	55.97 $\pm$ 0.47
With	52.59 $\pm$ 0.42	70.95 $\pm$ 0.51	60.41 $\pm$ 0.26

Table IV shows that class weighting is critical for handling TwiBot-22’s class imbalance, with F1 improving from 55.97 to 60.41 when enabled.

VI. DISCUSSION

The classifier ablation provides the clearest evidence: Logistic Regression, a linear classifier functionally equivalent to BotRGCN’s own softmax head, achieves 60.02 F1, exceeding end-to-end BotRGCN by 2.42 F1. This argues against the hypothesis that XGBoost’s non-linear capacity is the primary driver of these gains. Instead, the results point to the removal of competing gradient signals during GNN training as a likely contributing mechanism: when the GNN no longer needs to simultaneously optimize for structural representation and linear separability, it produces richer embeddings that any downstream classifier can exploit. The variance reduction from 0.78 to 0.26 further supports this interpretation. Decoupling stabilizes the gradient signal during representation learning, producing embeddings that are less sensitive to random initialization. GAT, which exhibited the highest end-to-end variance of ± 1.34 , benefits most from this stabilization with a reduction to ± 0.26 . A controlled comparison between end-to-end and decoupled training under identical class weighting settings has not been conducted and warrants further investigation. However, this study didn’t address the complete absence of classification signal during GNN pretraining; BotRGCN is still trained with its original softmax head. Nevertheless, the classifier ablation in Table II provides evidence against overfitting to the original head: Logistic Regression, functionally equivalent to BotRGCN’s softmax head, achieves 60.02 F1 on frozen embeddings, exceeding the end-to-end baseline by +2.42. While the proposed framework does not surpass HCGBot [13] (F1 = 62.25), the contribution is an orthogonal training strategy applicable to any GNN backbone rather than a novel architecture. The primary limitation of this study is that experiments are conducted on a single benchmark, TwiBot-22. Evaluation on additional datasets would strengthen the generalizability claim. Furthermore, the current framework relies on node-level embeddings, leaving temporal dynamics and community-level patterns unexplored.

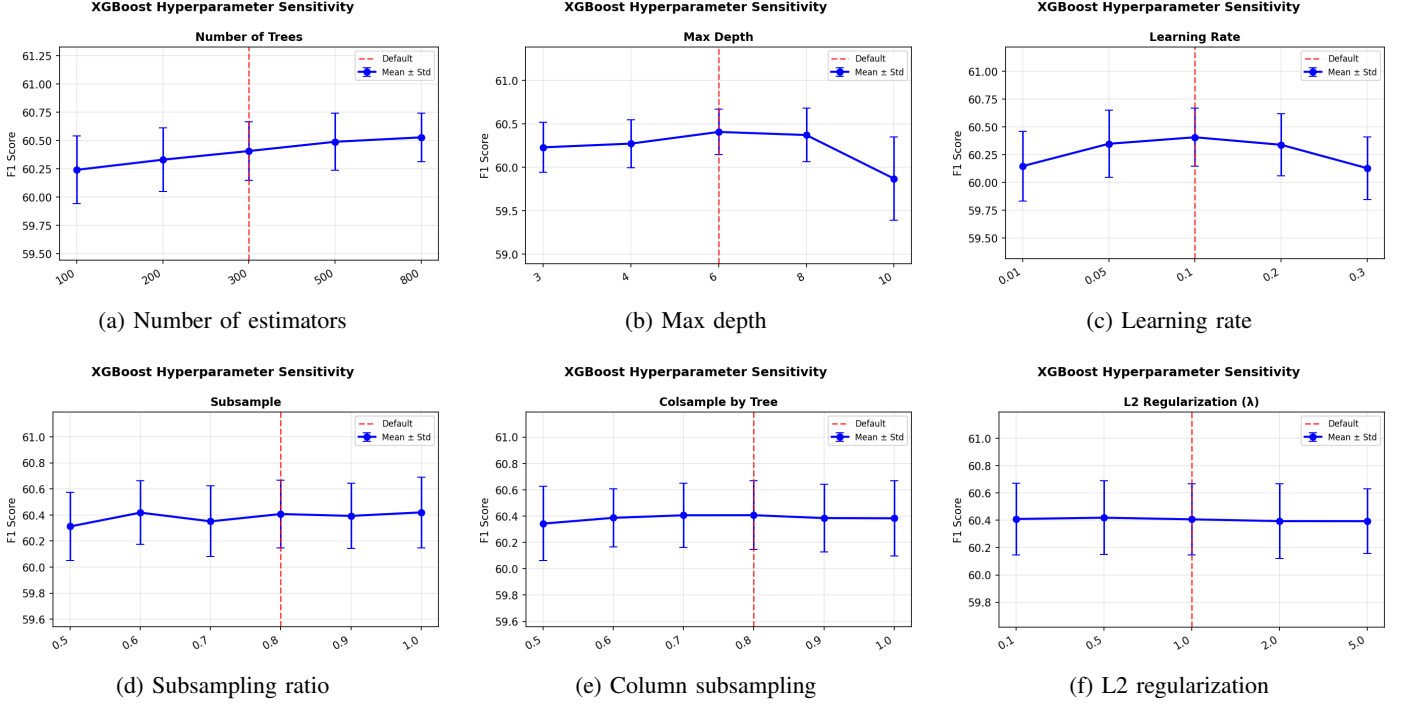


Fig. 2: XGBoost hyperparameter sweep results.

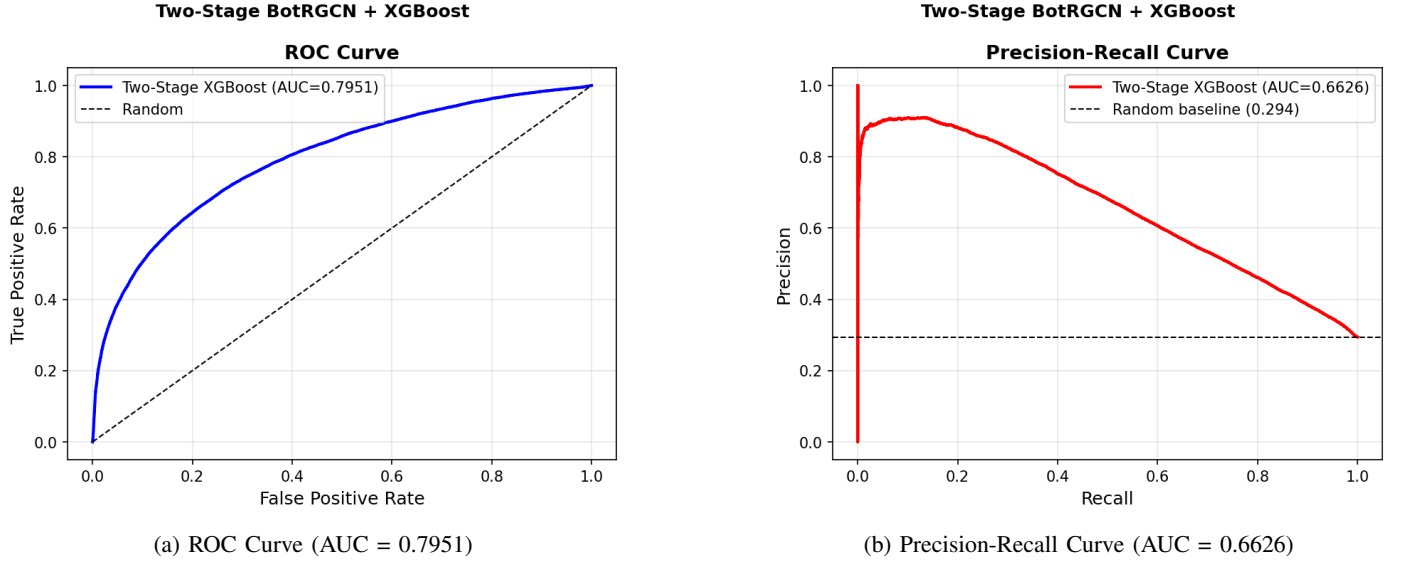


Fig. 3: ROC and Precision-Recall curves for the decoupled BotRCN + XGBoost framework.

VII. CONCLUSION

This study proposes a hybrid two-stage framework for Twitter bot detection that decouples GNN representation learning from classification. BotRCN serves as a graph-aware feature extractor, and XGBoost is trained independently on the frozen embeddings, a non-differentiable classifier only accessible through decoupling. Across 5 random seeds on TwiBot-22, the framework achieves F1 of 60.41, outperforming end-to-end BotRCN (57.60) by 2.81 F1 while reducing variance from 0.78 to 0.26. The improvement is consistent across GCN [10], GAT [11], and BotRCN [6], providing evidence that the bot-

tleneck may be attributable to coupling. Classifier ablation further shows that even Logistic Regression on frozen embeddings outperforms end-to-end training, suggesting that decoupling is the key contributing mechanism. Beyond accuracy gains, the proposed framework requires no architectural modifications to the underlying GNN and generalizes across GNN architectures without retuning. The second stage is lightweight, operating on frozen embeddings with a standalone classifier and requiring no gradient computation or backpropagation through the graph. Future work will explore LLM-based feature integration and feature alignment between GNN representations, language

TABLE V: Summary of main results consolidated across all experimental configurations.

Configuration	Precision	Recall	F1	Notes
<i>End-to-End Baselines</i>				
GAT end-to-end	68.92 ± 2.39	45.33 ± 2.71	54.58 ± 1.34	-
GCN end-to-end	61.83 ± 2.94	52.17 ± 3.67	56.40 ± 0.91	-
BotRGCN end-to-end	66.46 ± 1.46	50.89 ± 1.82	57.60 ± 0.78	-
<i>Decoupled Framework</i>				
GAT + XGBoost	49.31 ± 0.54	74.45 ± 0.50	59.32 ± 0.26	+4.74 F1 over end-to-end
GCN + XGBoost	49.75 ± 0.30	71.03 ± 0.90	58.51 ± 0.29	+2.11 F1 over end-to-end
BotRGCN + XGBoost	52.59 ± 0.42	70.95 ± 0.51	60.41 ± 0.26	+2.81 F1 over end-to-end
<i>Classifier Ablation</i>				
BotRGCN + RF	71.68 ± 0.49	42.35 ± 1.18	53.23 ± 0.86	Below end-to-end baseline
BotRGCN + MLP	67.32 ± 0.54	50.37 ± 1.32	57.61 ± 0.75	Comparable to end-to-end baseline
BotRGCN + LR	51.11 ± 0.55	72.72 ± 0.79	60.02 ± 0.40	Supports decoupling as mechanism
BotRGCN + XGBoost	52.59 ± 0.42	70.95 ± 0.51	60.41 ± 0.26	Best overall classifier
<i>Embedding Ablation</i>				
Last layer	52.19 ± 0.41	71.47 ± 0.63	60.33 ± 0.29	Layer selection has minimal impact
First layer	52.08 ± 0.44	71.79 ± 0.84	60.36 ± 0.21	Layer selection has minimal impact
All layers	52.59 ± 0.42	70.95 ± 0.51	60.41 ± 0.26	Default configuration

model encoders, and fully classifier-free pretraining.

ACKNOWLEDGMENT

This research was supported by The VNUHCM-University of Information Technology’s Scientific Research Support Fund.

DATA AVAILABILITY STATEMENT

All datasets used in this study are publicly available. TwiBot-22 is available via the official benchmark repository [7]. No proprietary data were used. The source code for this study is available at <https://github.com/SantiagoDeStefano/hybrid-twitter-bot-detection-decoupling>.

REFERENCES

- [1] E. Ferrara, O. Varol, C. Davis, F. Menczer, and A. Flammini, “The rise of social bots,” *Communications of the ACM*, vol. 59, no. 7, p. 96–104, Jun. 2016. [Online]. Available: <http://dx.doi.org/10.1145/2818717>
- [2] C. Shao, G. L. Ciampaglia, O. Varol, K.-C. Yang, A. Flammini, and F. Menczer, “The spread of low-credibility content by social bots,” *Nature Communications*, vol. 9, no. 1, Nov. 2018. [Online]. Available: <http://dx.doi.org/10.1038/s41467-018-06930-7>
- [3] A. Bessi and E. Ferrara, “Social bots distort the 2016 u.s. presidential election online discussion,” *First Monday*, vol. 21, 11 2016.
- [4] O. Varol, E. Ferrara, C. A. Davis, F. Menczer, and A. Flammini, “Online human-bot interactions: Detection, estimation, and characterization,” 2017. [Online]. Available: <https://arxiv.org/abs/1703.03107>
- [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” 2018. [Online]. Available: <https://arxiv.org/abs/1706.02216>
- [6] S. Feng, H. Wan, N. Wang, and M. Luo, “Botrgcn: Twitter bot detection with relational graph convolutional networks,” in *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, ser. ASONAM ’21. New York, NY, USA: Association for Computing Machinery, 2022, p. 236–239. [Online]. Available: <https://doi.org/10.1145/3487351.3488336>
- [7] S. Feng, Z. Tan, H. Wan, N. Wang, Z. Chen, B. Zhang, Q. Zheng, W. Zhang, Z. Lei, S. Yang, X. Feng, Q. Zhang, H. Wang, Y. Liu, Y. Bai, H. Wang, Z. Cai, Y. Wang, L. Zheng, Z. Ma, J. Li, and M. Luo, “TwiBot-22: Towards graph-based twitter bot detection,” 2022. [Online]. Available: <https://arxiv.org/abs/2206.04564>
- [8] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?” 2022. [Online]. Available: <https://arxiv.org/abs/2207.08815>
- [10] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [11] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [12] S. Cresci, “A decade of social bot detection,” *Communications of the ACM*, vol. 63, no. 10, p. 72–83, 2020. [Online]. Available: <http://dx.doi.org/10.1145/3409116>
- [13] H. Wan, M. Luo, J. Wang, X. Chang, and Q. Zheng, “HCGBot: Learning Homophilous Context Graphs for Twitter Bot Detection,” *IEEE Transactions on Knowledge & Data Engineering*, vol. 38, no. 03, pp. 1813–1825, Mar. 2026. [Online]. Available: <https://doi.ieeeecomputersociety.org/10.1109/TKDE.2026.3656720>
- [14] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, “Sign: Scalable inception graph neural networks,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.11198>
- [15] Y. Wang, J. Zhang, S. Guo, H. Yin, C. Li, and H. Chen, “Decoupling representation learning and classification for gnn-based anomaly detection,” in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1239–1248. [Online]. Available: <https://doi.org/10.1145/3404835.3462944>
- [16] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis, “Decoupling representation and classifier for long-tailed recognition,” 2020. [Online]. Available: <https://arxiv.org/abs/1910.09217>
- [17] B. Athiwaratkun and K. Kang, “Feature representation in convolutional neural networks,” 2015. [Online]. Available: <https://arxiv.org/abs/1507.02313>
- [18] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [19] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1703.06103>
- [20] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song, “Heterogeneous graph neural networks for malicious account detection,” 2020. [Online]. Available: <https://arxiv.org/abs/2002.12307>